

MATH70099 - Big Data: Statistical Scalability with PySpark

Coursework 1 - Report

CID: 06060027

Date: 16 February 2026

Question 1: Big Data Survey Bias Analysis

Summary of Bradley et al. (2021)

Bradley et al. (2021) challenge a common Big Data belief: that very large n is sufficient for accurate population inference. Using US COVID-19 vaccination uptake, they show that massive opt-in online surveys can deliver estimates with extremely small nominal sampling error while remaining substantially biased because the data generating process is unrepresentative of the target population.

Empirically, the authors compile 267 survey estimates of first-dose vaccination uptake over 9 January to 19 May 2021 and compare each to a benchmark based on CDC vaccination records. They study several survey sources, with particular attention to very large instruments such as the Delphi–Facebook COVID-19 Trends, Impact Survey (hundreds of thousands of responses per week) and the Census Household Pulse Survey (tens of thousands per wave). Despite their scale, these surveys systematically overestimate uptake, and the discrepancy persists through time. By late spring 2021, the largest online surveys overstate first-dose coverage by roughly 17-21 percentage points relative to the CDC benchmark. Because n is so large, the reported standard errors around these estimates can be tiny; the striking feature is therefore not imprecision, but systematic error.

The key technical contribution is to frame this failure using an explicit decomposition of estimation error into a data quantity term and a data quality term, building on the “Big Data paradox” perspective. For a generic population mean, the total error can be expressed as a product of: (i) a representativeness component capturing the correlation between the outcome and the response propensity (or inclusion indicator), and (ii) a variance component that shrinks with n (or, more precisely, with an effective sample size). In a probability sample, the correlation between inclusion and the outcome is (approximately) zero by design, so increasing n reduces error. In an opt-in survey, inclusion depends on self-selection, and if response propensity is positively correlated with vaccination status, the representativeness component is non-zero and induces bias that does not vanish as n grows. Hence, larger datasets can reduce variance while leaving bias unchanged, producing overconfident inference.

The central “data quality” quantity emphasised by Bradley et al. is the **data defect correlation (ddc)**, i.e. the correlation between the outcome and the inclusion/response indicator. In their decomposition, the error can scale like $\text{ddc} \times \sqrt{(N - n)/n}$, so a huge n does not guarantee small error when representativeness is poor.

The paper further shows why standard post-survey adjustments only partially help. Weighting and post-stratification aim to remove bias by enforcing agreement with population margins for observed covariates (e.g., age, sex, education, geography). However, these procedures only guarantee unbiasedness if, conditional on the weighting variables, participation is independent of the outcome (a missing-at-random type assumption). In this application, there are plausible unobserved factors - health engagement, trust, political attitudes, compliance behaviour - that affect both response propensity and vaccination. If these factors are not fully proxied by the weighting variables, the conditional independence assumption fails and residual bias remains. Bradley et al. empirically demonstrate that reweighting reduces but does not eliminate the gap to the CDC series, consistent with selection on unobservables.

Beyond diagnosing bias, the authors provide a reusable workflow for Big Data quality control: benchmark against an external gold standard when available; quantify the magnitude of bias relative to nominal uncertainty; and interpret improvements from weighting through the lens of identifiability conditions (what must be true for the adjustment to succeed). A practical implication is that “effective information” can be far smaller than the raw n suggests: when representativeness is poor, huge convenience samples can contain little population-relevant signal.

The findings have direct implications for real-world Big Data applications. In public health, an upward bias of around 20 percentage points could mislead assessments of coverage and misdirect interventions. In platform-based analytics more broadly, participation and engagement mechanisms often correlate with outcomes of interest, so scale does not guarantee external validity. In machine learning, biased measurements translate into biased labels and covariate shift, leading to models that may look strong on observed users but fail systematically for underrepresented groups. Bradley et al. therefore argue for prioritising design and validation: triangulating against administrative/probability sources, auditing selection mechanisms, and reporting uncertainty that reflects selection-driven bias rather than only sampling variability.

Question 2: Poisson Matrix Factorization

Part (a): Statistical Query Model Analysis

Can Algorithm 1 be expressed within the Statistical Query Model framework?

Answer: No, Algorithm 1 cannot be expressed within the Statistical Query Model (SQM) framework without stepping outside the SQ abstraction.

Justification:

The Statistical Query Model, introduced by Kearns (1998), characterizes algorithms that interact with data only through (approximate) expectations of query functions, rather than computing and storing per-instance intermediate quantities.

Algorithm 1 violates the SQM requirements because it contains an explicitly **instance-level** step:

In particular, during Steps 3–6 the algorithm computes a *full responsibility vector* $\phi_{i,j,1:D}$ for **each** observed triplet (i, j, c_{ij}) and then normalises it so that $\sum_k \phi_{i,j,k} = 1$. Concretely, it forms quantities of the type

$$\phi_{i,j,k} \propto \exp(\psi(\lambda_{i,k}^u) - \log(\mu_{i,k}^u) + \psi(\lambda_{j,k}^v) - \log(\mu_{j,k}^v)), \quad k = 1, \dots, D.$$

This is the key reason it is not SQM: the SQ model allows interacting with data through a limited number of expectation queries, whereas Algorithm 1 requires *instance-specific* intermediate objects (the local responsibilities) that are conceptually maintained per observation. The subsequent “global” updates are indeed sums and look SQ-like, but they depend on these local $\phi_{i,j,k}$ values.

In other words, the aggregation steps are **SQ-like**, but the need to materialise $\phi_{i,j}$ per record (even transiently, during a pass over the data) goes beyond a pure SQ oracle model.

Practically, this is exactly why the algorithm is still a good fit for distributed systems like MapReduce/PySpark: you compute local ϕ per record and then reduce aggregate sufficient statistics. That implementation pattern is efficient in distributed computing, but it is different from the SQM oracle setting.

Therefore, while parts of Algorithm 1 can be written as aggregate queries, the presence of the per-observation local parameter update/normalisation step means Algorithm 1 is **not** strictly an SQM algorithm.

In conclusion algorithm 1 is not strictly SQM because it requires computing and normalising instance-level $\phi_{i,j,k}$ for each observation, which goes beyond accessing data only through a limited set of statistical queries.

Part (b): Hadoop MapReduce Implementation

This part implements Algorithm 1 (PMF coordinate ascent with variational updates) using **Hadoop Streaming**. At each iteration t :

1. The **mapper** reads triplets (i, j, c_{ij}) and the current parameters $(\lambda^u, \mu^u, \lambda^v, \mu^v)$ from the previous iteration.
2. It computes the Multinomial probabilities $\phi_{i,j,k}$ (Algorithm 1, Steps 5–6) and emits contributions $c_{ij} \phi_{i,j,k}$ keyed by either user or movie.
3. The **reducer** aggregates (sums) these contributions to obtain, for every user i and movie j , the sufficient statistics needed to update λ .
4. A separate Python script updates the remaining parameters (μ terms) that do not require iterating over ϕ explicitly.

To keep the μ updates scalable, we exploit their structure by first computing once per iteration the vectors $s_v[k] = \sum_j \lambda_{j,k}^v / \mu_{j,k}^v$ and $s_u[k] = \sum_i \lambda_{i,k}^u / \mu_{i,k}^u$, then broadcasting across users/movies: $\mu_{:,k}^u = b_u + s_v[k]$ and $\mu_{:,k}^v = b_v + s_u[k]$.

All scripts were placed on Athena in `~/cw1/q2b/`.

After execution, the working directory contains the iterative parameter snapshots and reducer outputs, e.g.:

```
lap125@athena:~/cw1/q2b$ ls
hadoop_output.txt  lam_u_1.npy  lam_u_4.npy  lam_v_1.npy  lam_v_4.npy  mu_u_0.npy  mu_u_3.npy  mu_v_0.npy  mu_v_3.npy
↪ reducer_output_iter0.txt  reducer_output_iter3.txt  update_parameters.py
hadoop_run.sh      lam_u_2.npy  lam_u_5.npy  lam_v_2.npy  lam_v_5.npy  mu_u_1.npy  mu_u_4.npy  mu_v_1.npy  mu_v_4.npy
↪ reducer_output_iter1.txt  reducer_output_iter4.txt
lam_u_0.npy        lam_u_3.npy  lam_v_0.npy  lam_v_3.npy  mapper.py    mu_u_2.npy  mu_u_5.npy  mu_v_2.npy  mu_v_5.npy
↪ reducer_output_iter2.txt  reducer.py
```

Q2b results:

Final estimates after 5 iterations (first 5 dimensions):

Users (IDs 42, 117, 194):

User ID	$\lambda_{i,1:5}^u$	$\mu_{i,1:5}^u$
42	[0.106647, 0.100011, 427.633333, 60.669749, 0.100021]	[1149.913871, 905.491722, 808.785594, 818.137403, 881.651648]
117	[56.097730, 0.100010, 1.704513, 31.002309, 0.100011]	[1149.913871, 905.491722, 808.785594, 818.137403, 881.651648]

User ID	$\lambda_{i,1:5}^u$	$\mu_{i,1:5}^u$
194	[578.971843, 75.712949, 0.100058, 10.050923, 0.576911]	[1149.913871, 905.491722, 808.785594, 818.137403, 881.651648]

Movies (IDs 500, 2026, 8888):

Movie ID	$\lambda_{j,1:5}^v$	$\mu_{j,1:5}^v$
500	[0.100011, 9.627879, 0.100016, 0.100012, 0.100021]	[162.051598, 55.496255, 36.966167, 34.361963, 25.256120]
2026	[139.266194, 22.537598, 0.100017, 0.100020, 1.769888]	[162.051598, 55.496255, 36.966167, 34.361963, 25.256120]
8888	[0.293685, 42.045841, 0.100001, 0.100023, 0.100015]	[162.051598, 55.496255, 36.966167, 34.361963, 25.256120]

Mapper Implementation (mapper.py):

```
#!/usr/bin/env python3
"""
Mapper for Poisson Matrix Factorization - Question 2b
Computes phi (Multinomial probabilities) for each observation.
"""

import sys
import numpy as np
from scipy.special import digamma

def load_parameters(iteration, N=610, M=9724):
    """Load current variational parameters from files."""
    if iteration == 0:
        # Load initial parameters
        lam_u = np.load('lam_u_0.npy')
        mu_u = np.load('mu_u_0.npy')
        mu_u = np.tile(mu_u, (N, 1)) ## Expand to (N, D)
        lam_v = np.load('lam_v_0.npy')
        mu_v = np.load('mu_v_0.npy')
        mu_v = np.tile(mu_v, (M, 1)) ## Expand to (M, D)
    else:
        # Load parameters from previous iteration
        lam_u = np.load(f'lam_u_{iteration}.npy')
        mu_u = np.load(f'mu_u_{iteration}.npy')
        lam_v = np.load(f'lam_v_{iteration}.npy')
        mu_v = np.load(f'mu_v_{iteration}.npy')

    return lam_u, mu_u, lam_v, mu_v

def main():
    """
    Mapper main function.

    Input: Each line from movie_counts.csv in format: userID,movieID,count
    Output: Key-value pairs for reducer
        - For user aggregation: U\tuserID>\t<k>\t<count*phi_k>
        - For movie aggregation: V\tmovieID>\t<k>\t<count*phi_k>
    """

    # Read current iteration number from environment or use 0 (we'll need it in hadoop_run.sh)
    import os
    current_iteration = int(os.environ.get('ITERATION', '0'))

    # Load current parameters
    lam_u, mu_u, lam_v, mu_v = load_parameters(current_iteration, N=610, M=9724)
    D = lam_u.shape[1] # Latent dimensions
```

```

# Process each line from stdin (one observation per line)
for raw in sys.stdin:
    line = raw.strip()

    # Parse input: userID,movieID,count
    parts = line.split(',')
    i = int(parts[0])
    j = int(parts[1])
    c_ij = int(parts[2])

    # Vectorised log-phi computation for numerical stability
    log_phi = (digamma(lam_u[i, :]) - np.log(mu_u[i, :]) +
               digamma(lam_v[j, :]) - np.log(mu_v[j, :]))

    # stabilize before exponentiating
    log_phi = log_phi - np.max(log_phi)
    phi = np.exp(log_phi)
    s = phi.sum()
    if s <= 0:
        phi = np.full(D, 1.0 / D)
    else:
        phi = phi / s

    contrib = c_ij * phi

    # Emit user and movie contributions using composite key so Hadoop groups by (type,id,k)
    for k, val in enumerate(contrib):
        print(f"U,{i},{k}\t{val}")
    for k, val in enumerate(contrib):
        print(f"V,{j},{k}\t{val}")

if __name__ == "__main__":
    main()

```

Reducer Implementation ([reducer.py](#)):

```

#!/usr/bin/env python3
"""
Reducer for Poisson Matrix Factorization - Question 2b
Aggregates phi values for user and movie embedding updates.

This reducer expects mapper output with a composite key before the first tab:
    TYPE,ID,K\t<value>
with value as a float and aggregates values for each (TYPE, ID, K) triple.
"""

import sys

def main():

    current_key = None
    current_sum = 0.0

    # Process each line from stdin (sorted by key)
    for line in sys.stdin:
        line = line.strip()

        # Expect mapper to emit: key\tvalue where key = "TYPE,ID,K"
        parts = line.split('\t', 1)
        key_str, value_str = parts
        entity_type, entity_id_str, k_str = key_str.split(',')
        entity_id = int(entity_id_str)
        k = int(k_str)
        value = float(value_str)

        key = (entity_type, entity_id, k)

        # If this is a new key, output the previous aggregation
        if current_key is not None and key != current_key:
            print(f"{current_key[0]}\t{current_key[1]}\t{current_key[2]}\t{current_sum}")
            current_sum = 0.0

        current_key = key
        current_sum += value

    # Output the last aggregation
    if current_key is not None:
        print(f"{current_key[0]}\t{current_key[1]}\t{current_key[2]}\t{current_sum}")

if __name__ == "__main__":
    main()

```

Parameter update script ([update_parameters.py](#)):

```
#!/usr/bin/env python3
"""
Update variational parameters after MapReduce aggregation - Question 2b
This script runs after each MapReduce iteration to update the Gamma parameters.
"""

import sys
import numpy as np

def update_parameters_from_reducer_output(reducer_output_file, iteration,
                                         N=610, M=9724, D=10,
                                         a_u=0.1, b_u=0.1, a_v=0.1, b_v=0.1):
    """
    Update Gamma parameters based on reducer output.

    Args:
        reducer_output_file: Path to reducer output file
        iteration: Current iteration number (0-4)
        N: Number of users
        M: Number of movies
        D: Latent dimensions
        a_u, b_u, a_v, b_v: Gamma prior hyperparameters

    Usage: python update_parameters.py <reducer_output_file> <iteration>
    """

    # Load current parameters
    if iteration == 0:
        # Load initial parameters
        lam_u = np.load('lam_u_0.npy')
        mu_u = np.load('mu_u_0.npy')
        mu_u = np.tile(mu_u, (N, 1)) ## Expand to (N, D)
        lam_v = np.load('lam_v_0.npy')
        mu_v = np.load('mu_v_0.npy')
        mu_v = np.tile(mu_v, (M, 1)) ## Expand to (M, D)
    else:
        lam_u = np.load(f'lam_u_{iteration}.npy')
        mu_u = np.load(f'mu_u_{iteration}.npy')
        lam_v = np.load(f'lam_v_{iteration}.npy')
        mu_v = np.load(f'mu_v_{iteration}.npy')

    # Initialize aggregation dictionaries
    user_phi_sums = {} # {(i, k): sum_value}
    movie_phi_sums = {} # {(j, k): sum_value}

    # Read reducer output
    with open(reducer_output_file, 'r') as f:
        for line in f:
            line = line.strip()

            parts = line.split('\t')

            entity_type, entity_id, k, sum_value = parts
            entity_id = int(entity_id)
            k = int(k)
            sum_value = float(sum_value)

            if entity_type == 'U':
                user_phi_sums[(entity_id, k)] = sum_value
            elif entity_type == 'V':
                movie_phi_sums[(entity_id, k)] = sum_value

    # Update user parameters (lam_u, mu_u)
    # Update lambda_u efficiently
    lam_u[:, :] = a_u
    for (i, k), sum_phi in user_phi_sums.items():
        lam_u[i, k] = a_u + sum_phi

    # Update mu_u: compute s_v[k] = sum_j lam_v[j,k] / mu_v[j,k] once per iteration
    s_v = np.sum(lam_v / mu_v, axis=0) # shape (D,)
    mu_u[:, :] = b_u + s_v[None, :]

    # Update movie parameters (lam_v, mu_v)
    # Update lambda_v efficiently
    lam_v[:, :] = a_v
    for (j, k), sum_phi in movie_phi_sums.items():
        lam_v[j, k] = a_v + sum_phi
```

```

# Update mu_v: compute s_u[k] = sum_i lam_u[i,k] / mu_u[i,k] once per iteration
s_u = np.sum(lam_u / mu_u, axis=0) # shape (D,)
mu_v[:, :] = b_v + s_u[None, :]

# Save updated parameters
next_iteration = iteration + 1
np.save(f'lam_u_{next_iteration}.npy', lam_u)
np.save(f'mu_u_{next_iteration}.npy', mu_u)
np.save(f'lam_v_{next_iteration}.npy', lam_v)
np.save(f'mu_v_{next_iteration}.npy', mu_v)

print(f"Parameters updated and saved for iteration {next_iteration}")

if __name__ == "__main__":
    reducer_output = sys.argv[1]
    iteration = int(sys.argv[2])
    update_parameters_from_reducer_output(reducer_output, iteration)

```

Hadoop execution script (hadoop_run.sh):

```

#!/bin/bash
#####
# Hadoop MapReduce execution script for PMF Algorithm - Question 2b
# This script runs 5 iterations of the coordinate ascent algorithm
#####

# Set paths
DATA_FILE="/shared_data/2026/cw1/movie_counts.csv" # On HDFS
WORK_DIR="$HOME/cw1/q2b" # Working directory on local filesystem
# Output to /tmp to avoid permission issues
OUTPUT_BASE="/tmp/lap125/cw1/output"

# Create working directory if it doesn't exist
mkdir -p $WORK_DIR
cd $WORK_DIR

# Copy initial parameter files to working directory
cp /home/shared_data/2026/cw1/lam_u_0.npy .
cp /home/shared_data/2026/cw1/lam_v_0.npy .
cp /home/shared_data/2026/cw1/mu_u_0.npy .
cp /home/shared_data/2026/cw1/mu_v_0.npy .

echo "Starting PMF Algorithm with Hadoop MapReduce"
echo "Data file: $DATA_FILE"
echo "Working directory: $WORK_DIR"
echo ""

# Run 5 iterations
for ITER in {0..4}
do
    echo "Iteration $((ITER + 1))/5"
    echo "Current iteration: $ITER"

    OUTPUT_DIR="${OUTPUT_BASE}_iter${ITER}"

    # Remove output directory if it exists
    hdfs dfs -rm -r $OUTPUT_DIR 2>/dev/null

    # Set iteration number in environment variable for mapper
    export ITERATION=$ITER

    # Run Hadoop MapReduce job
    echo "Running Hadoop streaming job"
    hadoop jar /usr/local/hadoop-3.3.4/share/hadoop/tools/lib/hadoop-streaming-3.3.4.jar \
        -D mapreduce.job.name="PMF_Iteration_${(ITER+1)}" \
        -files mapper.py, reducer.py, lam_u_${ITER}.npy, lam_v_${ITER}.npy, mu_u_${ITER}.npy, mu_v_${ITER}.npy \
        -cmdenv ITERATION=$ITER \
        -mapper "python3 mapper.py" \
        -reducer "python3 reducer.py" \
        -input $DATA_FILE \
        -output $OUTPUT_DIR

    # Copy reducer output back to local filesystem
    REDUCER_OUTPUT="reducer_output_iter${ITER}.txt"
    echo "Copying reducer output to local filesystem"
    hdfs dfs -cat ${OUTPUT_DIR}/part-* > $REDUCER_OUTPUT

    # Update parameters
    echo "Updating parameters"
    python3 update_parameters.py $REDUCER_OUTPUT $ITER

```

```

    if [ $? -ne 0 ]; then
        echo "ERROR: Parameter update failed at iteration $((ITER + 1))"
        exit 1
    fi

    echo "Iteration $((ITER + 1)) completed successfully"
    echo ""
done

echo "All 5 iterations completed successfully"

# Display final results
echo "Final Results (first 5 dimensions):"

python3 << EOF
import numpy as np

# Load final parameters
lam_u_5 = np.load('lam_u_5.npy')
mu_u_5 = np.load('mu_u_5.npy')
lam_v_5 = np.load('lam_v_5.npy')
mu_v_5 = np.load('mu_v_5.npy')

print("\nUser Parameters:")
print("-" * 80)
for user_id in [42, 117, 194]:
    print(f"\nUser ID: {user_id}")
    print(f"    lambda_u: {lam_u_5[user_id, :5]}")
    print(f"    mu_u:      {mu_u_5[user_id, :5]}")

print("\n" + "-" * 80)
print("\nMovie Parameters:")
print("-" * 80)
for movie_id in [500, 2026, 8888]:
    print(f"\nMovie ID: {movie_id}")
    print(f"    lambda_v: {lam_v_5[movie_id, :5]}")
    print(f"    mu_v:      {mu_v_5[movie_id, :5]}")

print("\n" + "=" * 80)
EOF

echo "All files are located in: $WORK_DIR"

```

Part (c): PySpark RDD Implementation

Spark Solution Code ([coursework_solution.py](#)):

```

"""
MATH70099 - Coursework 1 - answer to Question 2c, 2d, and 2e
Compatible with Athena cluster execution
Author: 06060027
"""

import numpy as np
from scipy.special import digamma
try:
    from pyspark.sql import SparkSession
    from pyspark import SparkContext
    from pyspark.ml import Pipeline
    from pyspark.ml.classification import (
        LogisticRegression,
        RandomForestClassifier,
        MultilayerPerceptronClassifier,
    )
    from pyspark.ml.feature import VectorAssembler, StandardScaler, StringIndexer
    from pyspark.ml.evaluation import MulticlassClassificationEvaluator
except ModuleNotFoundError as e:
    raise ModuleNotFoundError(
        "PySpark is required to run Q2c/Q2d/Q2e. Install `pyspark` or run this script on Athena."
    ) from e
import os
import json

# Question 2c: PySpark RDD Implementation of PMF

```



```

def pyspark_pmf_algorithm(data_path, lam_u_0, lam_v_0, mu_u_0, mu_v_0,
                          N=610, M=9724, D=10, a_u=0.1, b_u=0.1,
                          a_v=0.1, b_v=0.1, n_iterations=5):
    """
    Implements Poisson Matrix Factorization using PySpark RDD operations.

    Args:
        data_path: Path to movie_counts.csv
        lam_u_0, lam_v_0, mu_u_0, mu_v_0: Initial variational parameters
        N: Number of users (610)
        M: Number of movies (9724)
        D: Latent dimensions (10)
        a_u, b_u, a_v, b_v: Gamma prior hyperparameters (0.1)
        n_iterations: Number of iterations to run (5)

    Returns:
        Final variational parameters after n_iterations
    """

    # Create SparkContext
    sc = SparkContext.getOrCreate()

    # Load data into RDD from HDFS or local filesystem
    # Each line: userID,movieID,count
    data_rdd = sc.textFile(data_path)

    # Parse data: (i, j, c_ij)
    parsed_data = data_rdd.map(lambda line: line.strip().split(',')) \
        .map(lambda x: (int(x[0]), int(x[1]), int(x[2]))) \
        .cache()

    # Initialize parameters
    lam_u = lam_u_0.copy()
    lam_v = lam_v_0.copy()

    # Initialize mu_u and mu_v as matrices (Athena initial files are vectors of length D)
    mu_u = mu_u_0.copy()
    mu_v = mu_v_0.copy()
    if mu_u.ndim == 1:
        mu_u = np.tile(mu_u, (N, 1))
    if mu_v.ndim == 1:
        mu_v = np.tile(mu_v, (M, 1))

    print(f"Starting PMF algorithm with {n_iterations} iterations")
    print(f"Data: N={N} users, M={M} movies, D={D} latent dimensions")

    for iteration in range(n_iterations):
        print(f"\n Iteration {iteration + 1}/{n_iterations}")

        # Broadcast current parameters to all workers
        bc_lam_u = sc.broadcast(lam_u)
        bc_mu_u = sc.broadcast(mu_u)
        bc_lam_v = sc.broadcast(lam_v)
        bc_mu_v = sc.broadcast(mu_v)

        # Step 1: compute per-observation contributions c_ij * phi_ijk and aggregate with reduceByKey
        def _compute_phi_vec(i: int, j: int) -> np.ndarray:
            lam_u_local = bc_lam_u.value
            mu_u_local = bc_mu_u.value
            lam_v_local = bc_lam_v.value
            mu_v_local = bc_mu_v.value

            log_phi = (
                digamma(lam_u_local[i, :])
                - np.log(mu_u_local[i, :])
                + digamma(lam_v_local[j, :])
                - np.log(mu_v_local[j, :])
            )
            # Stabilise exponentiation
            log_phi = log_phi - np.max(log_phi)
            phi = np.exp(log_phi)
            phi_sum = float(np.sum(phi))
            if phi_sum <= 0.0 or not np.isfinite(phi_sum):
                return np.ones(D) / D
            return phi / phi_sum

        def emit_contribs(record):
            i, j, c_ij = record
            phi = _compute_phi_vec(i, j)
            contrib = c_ij * phi
            # Tag keys so we can aggregate users and movies in one pass
            return [("U", i), contrib], [("V", j), contrib]

```

```

def add_vec(a, b):
    return a + b

stats_rdd = parsed_data.flatMap(emit_contribs).reduceByKey(add_vec).cache()

# Collect sufficient statistics (size: N*D + M*D)
user_sums = (
    stats_rdd.filter(lambda kv: kv[0][0] == "U")
    .map(lambda kv: (kv[0][1], kv[1]))
    .collectAsMap()
)
movie_sums = (
    stats_rdd.filter(lambda kv: kv[0][0] == "V")
    .map(lambda kv: (kv[0][1], kv[1]))
    .collectAsMap()
)

# Step 2: Update user embeddings (lam_u, mu_u) - Algorithm 1, Steps 7-9
print("Updating user embeddings")
user_sum_matrix = np.zeros((N, D))
for i, vec in user_sums.items():
    if 0 <= i < N:
        user_sum_matrix[i, :] = vec
lam_u = a_u + user_sum_matrix
sum_lam_v_over_mu_v = np.sum(lam_v / mu_v, axis=0)
mu_u = b_u + np.tile(sum_lam_v_over_mu_v, (N, 1))

# Step 3: Update movie embeddings (lam_v, mu_v) - Algorithm 1, Steps 10-12
print("Updating movie embeddings")
movie_sum_matrix = np.zeros((M, D))
for j, vec in movie_sums.items():
    if 0 <= j < M:
        movie_sum_matrix[j, :] = vec
lam_v = a_v + movie_sum_matrix
sum_lam_u_over_mu_u = np.sum(lam_u / mu_u, axis=0)
mu_v = b_v + np.tile(sum_lam_u_over_mu_u, (M, 1))

# Unpersist to free memory
stats_rdd.unpersist()
bc_lam_u.unpersist()
bc_mu_u.unpersist()
bc_lam_v.unpersist()
bc_mu_v.unpersist()

print(f"Iteration {iteration + 1} completed")

parsed_data.unpersist()

return lam_u, mu_u, lam_v, mu_v

def extract_slices_dict(
    lam_u,
    mu_u,
    lam_v,
    mu_v,
    user_ids=(42, 117, 194),
    movie_ids=(500, 2026, 8888),
    first_k=5,
):
    users = {}
    for uid in user_ids:
        users[str(uid)] = {
            "lambda_u_first5": lam_u[uid, :first_k].tolist(),
            "mu_u_first5": mu_u[uid, :first_k].tolist(),
        }
    movies = {}
    for mid in movie_ids:
        movies[str(mid)] = {
            "lambda_v_first5": lam_v[mid, :first_k].tolist(),
            "mu_v_first5": mu_v[mid, :first_k].tolist(),
        }
    return {"users": users, "movies": movies}

def extract_and_print_results(lam_u, mu_u, lam_v, mu_v,
    user_ids=[42, 117, 194],
    movie_ids=[500, 2026, 8888],
    first_k=5):
    """
    Extract and print results for specified users and movies.
    Returns: Formatted string of results.
    """
    output = ""

```

```

output += "FINAL RESULTS (Q2c: same users/movies as Q2b)\n"

output += "\nUser Parameters (first 5 dimensions):\n"
for user_id in user_ids:
    output += f"\nUser ID: {user_id}\n"
    output += f"    lambda_u: {lam_u[user_id, :first_k]}\n"
    output += f"    mu_u:      {mu_u[user_id, :first_k]}\n"

output += "\nMovie Parameters (first 5 dimensions):\n"
for movie_id in movie_ids:
    output += f"\nMovie ID: {movie_id}\n"
    output += f"    lambda_v: {lam_v[movie_id, :first_k]}\n"
    output += f"    mu_v:      {mu_v[movie_id, :first_k]}\n"

print(output)
return output

# Question 2d: Average Log-Likelihood Calculation

def compute_average_log_likelihood(data_path, lam_u, mu_u, lam_v, mu_v,
                                  N=610, M=9724):
    """
    Compute average log-likelihood loss using RDD operations.

    Formula:
     $L(\hat{U}, \hat{V}) = 1/(NM) * [\sum_{i,j} X_{ij} \log(u_i^T v_j) - \log(X_{ij}!) - \sum_i u_i^T * \sum_j v_j]$ 
    """
    sc = SparkContext.getOrCreate()

    # Compute u_hat and v_hat (means of variational posteriors)
    u_hat = lam_u / mu_u # Shape: (N, D)
    v_hat = lam_v / mu_v # Shape: (M, D)

    # Load data
    data_rdd = sc.textFile(data_path) \
        .map(lambda line: line.strip().split(',')) \
        .map(lambda x: (int(x[0]), int(x[1]), int(x[2])))

    # Broadcast parameters
    bc_u_hat = sc.broadcast(u_hat)
    bc_v_hat = sc.broadcast(v_hat)

    from scipy.special import gammaln

    # Compute first term:  $\sum_{i,j} [X_{ij} * \log(u_i^T v_j) - \log(X_{ij}!)]$ 
    def compute_observation_term(record):
        i, j, c_ij = record
        u_i = bc_u_hat.value[i]
        v_j = bc_v_hat.value[j]

        # Compute  $u_i^T v_j$ 
        u_dot_v = np.dot(u_i, v_j)

        # Avoid log(0)
        if u_dot_v <= 0:
            u_dot_v = 1e-10

        log_factorial = gammaln(c_ij + 1)

        return c_ij * np.log(u_dot_v) - log_factorial

    first_term = data_rdd.map(compute_observation_term).sum()

    # Compute second term:  $(\sum_i u_i) \cdot (\sum_j v_j)$  using RDD operations
    sum_u = (
        sc.parallelize(range(N), numSlices=min(64, N))
        .map(lambda i: bc_u_hat.value[i])
        .reduce(lambda a, b: a + b)
    )
    sum_v = (
        sc.parallelize(range(M), numSlices=min(64, M))
        .map(lambda j: bc_v_hat.value[j])
        .reduce(lambda a, b: a + b)
    )
    second_term = float(np.dot(sum_u, sum_v))

    # Compute average log-likelihood
    avg_log_likelihood = (1.0 / (N * M)) * (first_term - second_term)

    bc_u_hat.unpersist()
    bc_v_hat.unpersist()

    return avg_log_likelihood

```

```

# Question 2e: Genre Classification with PySpark ML

def classify_movie_genres(genres_data_path, test_size=0.2, random_seed=42):
    """
    Build and compare three classification models for movie genre prediction.

    Args:
        genres_data_path: Path to movie_genres.csv
        test_size: Proportion of data for testing (default: 0.2)
        random_seed: Random seed for reproducibility (default: 42)

    Returns:
        Dictionary with results for each model
    """

    # Initialize Spark Session
    spark = SparkSession.builder \
        .appName("Movie Genre Classification") \
        .getOrCreate()

    # Load data as DataFrame
    df = spark.read.option('header', True).option('inferSchema', True).csv(genres_data_path)

    # Remove missing observations (following Lecture 5 style)
    df = df.na.drop()

    print(f"Loaded {df.count()} movies with {len(df.columns)} columns")
    print("Schema:")
    df.printSchema()

    # Show sample
    print("\nSample data:")
    df.show(5, truncate=False)

    # Feature columns (all except 'movie' and 'genre')
    feature_cols = [col for col in df.columns if col not in ['movie', 'genre']]
    print(f"\nFeature columns: {feature_cols}")

    # Transform the DataFrame to have a suitable column 'features'
    vectorAssembler = VectorAssembler(inputCols=feature_cols, outputCol="features_raw")

    # Standard scaler for normalization
    scaler = StandardScaler(inputCol="features_raw", outputCol="features",
                            withStd=True, withMean=True)

    # INDEXING: Convert string 'genre' to numeric 'label'
    indexer = StringIndexer(inputCol="genre", outputCol="label")
    indexer_model = indexer.fit(df)
    df_indexed = indexer_model.transform(df)
    num_classes = len(indexer_model.labels)
    print(f"Number of genre classes: {num_classes}")

    # Split into training and test set
    train_df, test_df = df_indexed.randomSplit([1-test_size, test_size], seed=random_seed)

    print(f"\nTrain set: {train_df.count()} samples")
    print(f"Test set: {test_df.count()} samples")

    # Define three classification models
    # NOTE: All models now use labelCol="label" (numeric) instead of "genre"

    # Model 1: Logistic Regression
    lr = LogisticRegression(featuresCol="features", labelCol="label",
                            maxIter=100, regParam=0.01)
    pipeline_lr = Pipeline(stages=[vectorAssembler, scaler, lr])

    # Model 2: Random Forest
    rf = RandomForestClassifier(featuresCol="features", labelCol="label",
                               numTrees=100, maxDepth=10, seed=random_seed)
    pipeline_rf = Pipeline(stages=[vectorAssembler, scaler, rf])

    # Model 3: Multilayer Perceptron (Neural Network)
    # Input: 10 features, Hidden layers: [20, 15], Output: num_classes
    layers = [10, 20, 15, num_classes]
    mlp = MultilayerPerceptronClassifier(featuresCol="features", labelCol="label",
                                          layers=layers, maxIter=100, seed=random_seed)
    pipeline_mlp = Pipeline(stages=[vectorAssembler, scaler, mlp])

    # Train and evaluate models
    models = {
        "Logistic Regression": pipeline_lr,
        "Random Forest": pipeline_rf,
        "Multilayer Perceptron": pipeline_mlp
    }

```

```

}

results = {}

# Evaluators
# NOTE: Uses "label" (numeric) as ground truth
accuracy_evaluator = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction", metricName="accuracy"
)
f1_evaluator = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction", metricName="f1"
)

print("MODEL TRAINING AND EVALUATION")

for model_name, pipeline in models.items():
    print(f"\n {model_name}")

    # Train model
    print("Training")
    model = pipeline.fit(train_df)

    # Make predictions on test set
    print("Making predictions on test set")
    predictions = model.transform(test_df)

    # Evaluate
    accuracy = accuracy_evaluator.evaluate(predictions)
    f1_score = f1_evaluator.evaluate(predictions)

    print(f"Accuracy: {accuracy:.4f}")
    print(f"F1 Score: {f1_score:.4f}")

    results[model_name] = {
        "model": model,
        "accuracy": accuracy,
        "f1_score": f1_score,
        "predictions": predictions
    }

print("SUMMARY OF RESULTS")
print(f"{'Model':<30} {'Accuracy':<15} {'F1 Score':<15}")
for model_name, result in results.items():
    print(f"{'model_name':<30} {'result['accuracy']':<15.4f} {'result['f1_score']':<15.4f}")

return results

# Main Execution

def main():
    """Main function to run all parts."""

    # Configuration for Athena execution:

    # Input paths (read-only shared data)
    base_path_input = "/home/shared_data/2026/cw1" # Path to initial .npz files on local FS
    data_path = "/shared_data/2026/cw1/movie_counts.csv" # HDFS path for movie counts
    genres_path = "/shared_data/2026/cw1/movie_genres.csv" # HDFS path for genres

    # Output path (current directory)
    output_path = "."

    # Load initial parameters from shared directory
    print(f"Loading initial parameters from {base_path_input}")
    lam_u_0 = np.load(os.path.join(base_path_input, "lam_u_0.npz"))
    lam_v_0 = np.load(os.path.join(base_path_input, "lam_v_0.npz"))
    mu_u_0 = np.load(os.path.join(base_path_input, "mu_u_0.npz"))
    mu_v_0 = np.load(os.path.join(base_path_input, "mu_v_0.npz"))

    print("Initial parameter shapes:")
    print(f" lam_u_0: {lam_u_0.shape}")
    print(f" lam_v_0: {lam_v_0.shape}")
    print(f" mu_u_0: {mu_u_0.shape}")
    print(f" mu_v_0: {mu_v_0.shape}")

    # Question 2c: Run PMF algorithm with PySpark
    print("QUESTION 2c: PMF with PySpark RDD")

    lam_u_5, mu_u_5, lam_v_5, mu_v_5 = pyspark_pmf_algorithm(
        data_path, lam_u_0, lam_v_0, mu_u_0, mu_v_0,
        N=610, M=9724, D=10, n_iterations=5
    )

```

```

# Save final parameters
np.save(os.path.join(output_path, "lam_u_5_spark.npy"), lam_u_5)
np.save(os.path.join(output_path, "lam_v_5_spark.npy"), lam_v_5)
np.save(os.path.join(output_path, "mu_u_5_spark.npy"), mu_u_5)
np.save(os.path.join(output_path, "mu_v_5_spark.npy"), mu_v_5)
print("\nFinal parameters saved to .npy files")

# Extract and print results for the same users/movies as in Q2b
results_str = ""
results_str += extract_and_print_results(
    lam_u_5,
    mu_u_5,
    lam_v_5,
    mu_v_5,
    user_ids=[42, 117, 194],
    movie_ids=[500, 2026, 8888],
    first_k=5,
)

# Save slices in JSON for easy report inclusion
slices = extract_slices_dict(lam_u_5, mu_u_5, lam_v_5, mu_v_5)
with open("q2c_final_slices.json", "w") as f:
    json.dump(slices, f, indent=2)

# Question 2d: Compute average log-likelihood
print("QUESTION 2d: Average Log-Likelihood")

avg_ll = compute_average_log_likelihood(
    data_path, lam_u_5, mu_u_5, lam_v_5, mu_v_5
)

ll_str = f"\nAverage Log-Likelihood (Q2d): {avg_ll:.6f}\n"
print(ll_str)
results_str += ll_str

with open("q2d_avg_loglikelihood.json", "w") as f:
    json.dump({"average_log_likelihood": float(avg_ll)}, f, indent=2)

# Save text results to file
with open("results.txt", "w") as f:
    f.write(results_str)

# Question 2e: Genre classification
print("QUESTION 2e: Movie Genre Classification")

classification_results = classify_movie_genres(genres_path)

print("DONE, all results saved to results.txt and .npy files")

if __name__ == "__main__":
    main()

```

Results:

Q2c parameter extracts (same users/movies as Q2b; first 5 dimensions):

Final estimates after 5 iterations (first 5 dimensions):

Users (IDs 42, 117, 194):

	User ID	$\lambda_{i,1:5}^u$	$\mu_{i,1:5}^u$
	42	[0.106647, 0.100011, 427.633333, 60.669749, 0.100021]	[1149.913871, 905.491722, 808.785594, 818.137403, 881.651648]
	117	[56.097730, 0.100010, 1.704513, 31.002309, 0.100011]	[1149.913871, 905.491722, 808.785594, 818.137403, 881.651648]
	194	[578.971843, 75.712949, 0.100058, 10.050923, 0.576911]	[1149.913871, 905.491722, 808.785594, 818.137403, 881.651648]

Movies (IDs 500, 2026, 8888):

Movie ID	$\lambda_{j,1:5}^v$	$\mu_{j,1:5}^v$
500	[0.100011, 9.627879, 0.100016, 0.100012, 0.100021]	[162.051598, 55.496255, 36.966167, 34.361963, 25.256120]
2026	[139.266194, 22.537598, 0.100017, 0.100020, 1.769888]	[162.051598, 55.496255, 36.966167, 34.361963, 25.256120]
8888	[0.293685, 42.045841, 0.100001, 0.100023, 0.100015]	[162.051598, 55.496255, 36.966167, 34.361963, 25.256120]

Part (d): Average Log-Likelihood (RDDs)

Final result:

$$\bar{\ell}(\hat{U}, \hat{V}) = -0.156895$$

Part (e): Genre Classification

Model	Accuracy	F1 Score
Logistic Regression	0.2691	0.1749
Random Forest	0.3006	0.2157
Multilayer Perceptron	0.2782	0.1965

Interpretation:

The Random Forest model achieved the highest performance with an accuracy of 30.06% and an F1 score of 0.2157. The Multilayer Perceptron followed closely with 27.82% accuracy, while Logistic Regression performed the worst (26.91%).

Since the task has $K = 20$ classes, a random baseline accuracy is $1/K = 0.05$, so these results are well above chance even though the problem is challenging. Metrics are computed on a held-out test set using Spark’s `MulticlassClassificationEvaluator`.

The overall low accuracy across all models suggests that predicting movie genres solely from the latent factors (derived from user ratings) is a difficult task. The latent factors capture user preference patterns, which correlate with genre, but likely do not capture enough explicit content information to distinguish between 20 distinctive genres with high precision. However, the non-linear models (RF and MLP) slightly outperforming the linear model suggests there are complex interactions in the latent feature space.